2018 IEEE Confs on Internet of Things, Green Computing and Communications, Cyber, Physical and Social Computing, Smart Data, Blockchain, Computer and Information Technology, Congress on Cybermatics

Analyzing Financial Smart Contracts for Blockchain

Muskan Vinayak, Har Amrit Pal Singh Panesar, Saulo dos Santos,
Ruppa K. Thulasiram, Parimala Thulasiraman, S.S. Appadoo*
Department of Computer Science
*Department of Supply Chain Management, Asper School of Business
University of Manitoba
Winnipeg, Manitoba, Canada
{vinayakm, panehaps, dossants, tulsi, thulasir}@cs.umanitoba.ca; ss.appadoo@umanitoba.ca

***Abstract*—**
The recent advancement in Blockchain technology and cryptocurrencies like Bitcoin and Ethereum has captured interest of many researchers in academia and industry. Since its advent, the Blockchain was inherently supposed to be used in combination with cryptocurrencies but currently this technology is being used in other areas as well. One particular area based on the distributed ledger technology that has grabbed the attention of many technologist and financial marketers is "Smart Contracts". Based on a cryptocurrency framework known as Ethereum, these smart contracts can be used for different applications such as Option Pricing, Currency Exchange, Revenue Management System, Crowd-funding and Peer-to-Peer networking. In our current effort, we have designed a smart contract and demonstrate that this smart contract could be used to take various possible positions in an European style option. We have analyzed the contract for potential security vulnerabilities when implemented in Blockchain. These option based smart contracts could be used for collateral contract services among finance industries.

I. Introduction and Background

Blockchain is a decentralized ledger consisting of a series of blocks containing different transactions. This ledger can only be appended with the new blocks and acts as a source of trust between unknown parties. The concept was initiated in early 2009 by Satoshi Nakamoto [1] based on some prior work done by Vishnumurthy et al. [2].

One of the major impacts of Blockchain technology is seen on smart contracts that can be used to create user defined decentralized applications. Smart contracts are computer algorithms that define the rules of the execution of the service provided through these contracts. The idea of smart contracts is not new and was proposed by Nick Szabo [3] before even the introduction of bitcoin. Smart contract comprises of an algorithm, state of the contract, balance and logs. These smart contracts exist on Ethereum Blockchain. Ethereum [4] is a project through which decentralized transaction-based state machines can be developed. Ethereum blockchain is more generalized than bitcoin and stores Turing-complete programs. Ethereum stores these programs in the form of EVM (Ethereum Virtual Machine) bytecode [5]. The functionalities of these smart contracts can be utilized either internally by other contracts from within the blockchain or externally through transactions. To execute any computation on Ethereum, fee has to be paid in terms of gas, purchased using ether, the currency of Ethereum with Wei being the smallest sub-denomination [4]. This avoids the possibility of abusing the network [4]. Smart contracts can be explored to design some real world applications as decentralized applications such as option contract, credit default swap contract, voting system [6] [7] and independent applications for governance (for instance, outsourced computation [8], decentralized gambling [9].

A. Collateral Contract Services

A typical investment firm is comprised of Trading, Risk Management, Accounting/Reporting and Senior management team. The trading department is responsible for doing trades for the firm. They deal with other investment houses and exchanges. The nature of trades done is governed by the priorities of the investment firm in conjunction with the optimization of their portfolio. The Accounting/Reporting department is responsible for maintaining books of all the trades and prepares reports for board members and the shareholders. The Senior management team's task is to administrator the overall functioning of the firm. The collateral services contracts are handled manually in financial institutions these days and making them smart demands handling security issues. We address two major founding principle areas in this study: financial contracts and Blockchain technology. The key objectives of this research are (1) use a pool of multiple securities comprising of financial instruments, especially, derivatives; (2) formulate different sample Collateral Contract Services (CCS) each with different specifications; (3) algorithm design in identifying and selecting most appropriate instrument for the CCS and trade it in an automated manner; and (4) appropriate use of Blockchain technology in achieving these objectives.

B. Financial options contract

Our main focus is to study European Options using smart contracts. An option is an agreement between two parties, where the holder gets the right to exercise the option at a specific time (maturity) and the writer is obliged to the decision of the holder. The goal of option pricing is finding the *worth* of a contract. Five basic parameters that influence the price of an option are: underlying stock price *(S)*, strike price *(K)*, interest rate *(r)*, volatility (σ) and time until expiration *(T)*. For further description of options and other basic models (such as BSM, binomial lattice model, Monte-Carlo simulation, etc.), the readers may refer to [10]. However, we have adapted the

978-1-5386-7975-3/18/$31.00 ©2018 IEEE
DOI 10.1109/Cybermatics_2018.2018.00284

way an option is handled in financial markets to remove the possibility of defaults and make it execute on the Ethereum without the need of centralized clearing party.

C. Vulnerabilities

These smart contracts are vulnerable to attack by malicious users. One of the reasons that motivates adversaries to attack the smart contracts is that the contract holds some amount of money that can be gained through attacks. There has been certain incidents in the past highlighting the malfunctioning of smart contracts. These include the unexpected execution of smart contracts or thousands of dollars value of virtual coins being locked away [11]. Atzei et al. [12] have presented a detailed survey of such possible vulnerabilities. These vulnerabilities might lead to a huge loss as seen in The DAO (Decentralized Autonomous Organization) bug, which caused loss of few million dollars [13]. Moreover, the smart contracts are permanent and irreversible once deployed on the blockchain. So, the possible vulnerabilities have to be studied in a contract before its actual deployment to reduce the risk of attacks. Oyente [11] works with the EVM (Ethereum Virtual Machine) bytecode rather than the high-level language used to write contracts. In our project, we are using the open source tool called Oyente [11] to study possible vulnerabilities in the adapted options smart contract that we created.

Rest of this paper developed as follows: In section II we present related work with focus on transactions improvement and smart contracts. In section III we describe our experimental environment and setup including two approaches that we implemented, results from one of which we discuss in detail. Further, we present our observations on the vulnerabilities of smart contracts on Blockchain in this section with different versions of the contract with and without data from real market. Based on these observations we arrive at some conclusions and present them in section IV.

II. Related Work

Blockchain technology is being researched to check the technology's applicability in the financial sector. Some of the areas being studied for using blockchain in finance include cost savings, management techniques, prediction, possible security vulnerabilities and data feeds (used to fetch data from outside world). Some key most recent works explored in each of these areas are elaborated below:

According to Schneider et al. [14], with the use of blockchain-based technology in the financial sector there could be huge savings annually which is tens of billions of dollars approximately. Moreover, these include annual savings of around $11-12 billion for just settlement of cash securities [14]. Another application of distributed ledger technology was studied by Moyano and Ross [15]. They proposed a system to reduce the cost and other inefficiencies involved in the know-your-customer (KYC) process by financial system. In their system, the redundancy for this process by various institutions is removed using distributed ledger technology. If a customer wants to work with multiple financial systems the KYC process is only done by one institution and shared with the others.

Numerous research explore the ways to use the Ethereum and smart contracts concept for both eliminating the need of centralized trusted party and reducing operational costs in various financial sectors. Muller et al. [6] , presented a management system for financial contracts through which the execution of the contracts can be automated on a blockchain. For representing the financial contracts, they utilized the modified form of the language by Bahr et al. [16] and showed examples of different contracts designed using this language. These examples include forward contract, barrier option contract, to name a few.

Bentov et al. [17] talked about the prediction market in which people gamble on the results of possible future events. They presented an idea of possible enhancements in the cryptocurrency to accommodate fully decentralized prediction market. For this, they employed a novel modification of concept of Colored coins [18]. In their work, they proposed that this concept can be extended to incorporate other financial instruments and have real-time exchange of cryptocurrencies without relying on the trusted third party.

Tools to fetch information about financial data include Oraclize [19] and Schelling coin [20]. Private and public instances of different blockchains can be combined with Oraclize. Oraclize is constantly monitoring the Ethereum blockchain for requests to fetch data. When it is received, Oraclize asynchronously fetches the results based on the parameters specified and sends the results in a transaction.

Eskandari et al. [21] developed a tool called *pricegeth* to fetch price and implemented a smart contract representing a customized derivative option. They deployed this contract on Ethereum and limits the loss that the party can incur. They adjusted their contract and made it fully collateralized [21] along with setting a limit on the loss incurred by the participants in the contract. Our idea of representing an option contract is similar to the one presented in this paper. *Pricegeth*, as highlighted by the authors [21] fetches external information about exchange prices. This is different from the other price fetching tools, such as Oraclize, as it allows user to access historical data. The *pricegeth* server continuously watches the *Ethereum testnet* to see if new blocks are added and whenever a new block is added, *pricegeth* fetches the exchange prices at that time [21].

Another important factor to consider while developing the smart contracts is its security. In other words, the possible vulnerabilities present in the contract that might be exploited by the adversaries for their gain. Bhargavan et al. [5] made an attempt to verify the smart contracts formally using a functional programming language called F*. In their proposed approach, functional precision and run-time security of the Ethereum smart contracts can be studied. A similar approach was adopted by Luu et al. [11] in which they tried to formalize the semantics for smart contracts. Interestingly, they were able to identify certain subtle concepts that helped them to design a vulnerability detection tool for smart contracts.

Smart contracts can be used to create decentralized application for various industries like finance, trading, payment system etc. A lot of work has been done recently regarding creation of smart contracts and addressing the associated challenges [7]. The scope and application of smart contracts is very wide in creating decentralized applications.

III. Experimental Results

In this section we present and discuss the experimental set up that we have organized for the study and analyze the smart contracts on Blockchain.

A. Experimental Setup

The collateral services of contracts (CCS) is a practical problem handled in financial institutions every day. The hypothesis for this research is to conceptualize and implement CCS in an efficient manner using smart contracts. For a large financial institution, deciding which of their securities to trade for end of the day collateral services to maintain the contracts with multiple other financial institutions, is a complex problem. This necessitates significant level of automation through smart contracts using Blockchain technologies, which captures past transactions (trading of securities on previous days). Our current effort addresses the way to achieve this goal and involves design of new contracts to start with and then to analyze and monitor them on a timely basis so that financial health of the institution is maintained.

Generating and managing smart contracts is a multistage process.

*1) **Creating Smart Contract**:* Ethereum is a Blockchain platform which we can use to write smart contracts. In this environment, a smart contract is a user-defined algorithm that runs on the Blockchain. Through this a user can encode the rules defining the terms and obligations (amount, deadline etc.) of the contract. For instance, on the occurrence of some event, the contract initiates the transfer of Ether. We design and develop decentralized application based on our smart contract to be executed on the Blockchain.

Therefore, we can use smart contracts to implement the financial derivatives using Blockchain and hence the dependency on the intermediate party can be avoided.

*2) **First Step: Creating Smart Contract for Financial Options**:* Our approach differs from one of the recent works [21] in that the contract created by them is a currency swap option whereas our contract is an actual European Option traded in finance market. Our contract is designed to include both put and call options. Various parameters are initialized that define the strike price, expiration date (in terms of number of blocks to be added before expiration), call and put premium required, and *pricegeth* address (to fetch currency exchange rate). The user interacting with the contract can either take the position of a holder or a writer in either type of the option. In our present contract, the underlying contract takes the opposite position. However, in future we will look into the ways of how two different parties who are willing to take the opposite positions in the contract can be entered into a contract with each other. This contract is mainly designed for speculators who want to benefit from the market movements in the future. The speculators expect the change in the price to be in their favor depending on their position in the contract.

When the user calls any function of the smart contract, contract creation is started. The writer of the contract is paid certain fixed amount of premium by the holder. Also, the writer of the contract pays certain amount that is locked into the contract and used at the time of expiration for clearing and settlement of the contract. In the contract, on the expiration time, the price of the currency exchange rate (BTCETH) is recorded and compared with the strike price. Then, depending upon the position of the user and the difference between strike and market price, the default amount initially paid is adjusted. However, smart contracts are passive and have to be triggered by the authorized end-user, the 'trading department' mentioned earlier, if they find appropriate to execute it. There is another difference between our option strategy and the options in finance market. In options market, the loss for the writer could be unlimited and trading of asset can also take place. However, in our contract the loss for writer is limited to the default amount provided by the writer at the time of contract creation and only the profit is paid to the holder, if any.

Environmental Setup: For implementing the smart contract on European Options, we have used the language called *Solidity* [22]. Solidity is a high-level language that is a lot similar to Javascript and is influenced by different languages including Javascript, Python and C++ [22]. This language is contract oriented and user-friendly. After the smart contracts are written, they are compiled into EVM bytecode and published on the Blockchain. As to write any program, a debugger is required. For this purpose, we have used Remix [23] Integrated Development Environment . Remix is an Integrated development environment with the main concentration on the development and testing by deployment of the smart contracts written in Solidity language [23]. This IDE provides different useful features which includes debugging and developing a smart contract, analysis of the code being written and warning the user about possible flaws, debugging any committed transaction, etc [23].

As an option is a derivative whose value depends upon the underlying asset which in our case is the exchange traded (BTCETH). However, at present the exchange is not decentralized so there is a need to fetch the data from external sources. This data will decide the actual transfer between the holder and the writer at the expiration time. As mentioned earlier, information feed that we have used in our study is *pricegeth* [21]. It is an Application Programming interface on Ethereum Blockchain [24]. The server of *pricegeth* constantly monitors the geth (Testnet Ropsten). Whenever it observes that a new block is added, the "Poloniex Prices" at that time are saved by the server. Entire historical prices are stored by the *pricegeth*. These prices are further sent to the *pricegeth* smart contract which can be queried by other contracts for price at specific block number [24].

We present below the pseudo-code for the generating and

monitoring of smart contract, which we describe following the pseudo-code. We have basically represented three main functions. The user can execute the buy call option method to be a buyer in the contract. For this the user need to pay the amount equal to the premium specified in the contract.

Algorithm 1 *buyCallOption* method

```
Input: Sender's (Holder) Address, premium
Output: Allocated optionId
Required: Premium Sent = Option Premium Required
  Required: Contract's balance = Default Settlement Balance
  if Conditions Required Satisfied then
  |   Option Created and Initialized
  |     Premium is paid to the writer
  |     returns Option ID
end
else
  |   Premium is refunded
  |     Execution is not successful
end
```

Another position that a user can take is to act as a holder of the contract. In this case however, the user has to send the amount that will be used for settlement between the parties at the expiration time.

Algorithm 2 *writeCallOption* method

```
Input: Sender's (Writer) Address, Collateral Amount
Output: Allocated optionId
Required: Collateral Amount >= Min. Default Balance
  Required: Contract's balance = Premium Required
  if Conditions Required Satisfied then
  |   Option Created and Initialized
  |     Premium is paid to the writer
  |     returns Option ID
end
else
  |   Default balance is refunded
  |     Execution is not successful
end
```

The psedo code in algorithm 3 can be executed by the holder/buyer of any type of contract and receive the profit if any.

3) ***Second Step: Monitoring and Analyzing Smart Contracts****:*

a) First Approach: Using Oyente: : We analyzed our contracts using another software called Oyente [11]. The main idea is to identify the bugs in the Smart Contracts before they are deployed on the blockchain. According to the authors [11], out of 19336 existing Ethereum contracts at the time of their research, Oyente flagged 8833 of them as vulnerable.

b) Second Approach: Monitoring Smart Contracts using EtherBeat: Once a smart contract is deployed on the Blockchain, it is difficult to modify it and this results in hard-forks. Therefore, even if a developer identifies a critical bug it is not possible to fix the issue in a straight forward manner. So, in our initial attempt with this approach, we tried to deploy EtherBeat on our local machine to monitor smart contracts. EtherBeat [25] is an open source project based on Ethereum. According to its developers, this web application can be used to monitor contracts running on Blockchain. Along with safeguarding the contract's key functions like sending and receiving Ether, this service also allows interaction with and

Algorithm 3 *Exercise Option*

```
Input: Sender's Address
Output: Bool: Success/Fail
Required: Valid Sender Option ID
  Required: Contract not exercised before
  Required: Valid Exercise Time
  if Current Stock Price == 0 then
  |   Return False
end
else if Call Option then
  |   if Stock price > Strike price then
  |     |   Transfer profit and adjust balance
  |   end
  |   else
  |     |   Adjust balance
  |   end
end
else
  |   if Strike price > Stock price then
  |     |   Transfer profit and adjust balance
  |   end
  |   else
  |     |   Adjust balance
  |   end
end
```

visualization of its functions. This is also beneficial to other smart contracts which have any kind of dependency on the contract being monitored. At first, we installed the dependencies required for EtherBeat but still faced issues with execution of EtherBeat. Further work with EtherBeat is currently ongoing.

4) **Evaluation:** For compiling and debugging the smart contracts we have used an integrated development environment (IDE) called Remix, which provides an integrated debugging and testing environment.

B. Description of Various Vulnerabilities and Results

According to the authors in [11], there are several security bugs which allow malicious users or miners to exploit the contract and gain ethers. As mentioned earlier, once a contract is published on the Blockchain, it cannot be changed or altered. Therefore, there is a definite need to analyze the contract especially in option pricing because a lot of money is involved and a slight vulnerability can affect the market significantly.

The vulnerabilities identified by Oyente [11] are as follows:

1) Transaction - Ordering Dependency: We know that a block in the Blockchain contains a set of transactions. Now, within that block the order in which transactions are executed is a big factor in determining the final outcome. There are two main explanation as to why this vulnerability is so crucial:

a) Benign Scenario: Let us consider a situation in which a seller is trying to sell a contract to a buyer. Let T1 be a transaction in which the seller updates the selling price of a particular asset and T2 be a transaction in which the buyer tries to buy the underlying asset from the seller at a price before the update done in T1. Now, if these transactions are in the same block, then there exist chances that T1 is scheduled before T2. Therefore, the buyer will end up paying more than what s/he is expected to pay and may not get a chance to withdraw from entering the contract.

b) Malicious Attack: The previous case is an example in which the seller might not know whether someone has already tried to buy the asset and s/he is not maliciously trying to

gain profit. However, there exists another case in which the seller may try to benefit from this vulnerability. For instance, there exists a time gap between when a buyer broadcast the transaction to enter an option contract and the acceptance of the block in which that transaction is executed, specifically in Ethereum, as suggested in [11]. The time to find a new block is 12 sec. Thus, a malicious seller can keep observing the network to check if there exists a transaction T which tries to buy her/his underlying asset. When s/he observe such a transaction, s/he can immediately send a new Transaction T' which updates the price accordingly, such that the buyer has to pay more than the listed price. Although there is a probabilistic chance that T' will occur before T and if it actually does happen then the buyer will have to pay a lot more than the amount which s/he actually perceived at the time of entering into the contract. When one is trying to sell the asset, the reverse may happen.

2) Time Stamp Dependency: This vulnerability occurs if our smart contract uses the block time-stamp as a criterion to execute some critical operation(s).

A good example is when the option contract uses time-stamp to determine the price of a stock at a given interval of time. Now, according to [26], while mining the block, the miner has to set a value for time-stamp when the block is accepted. In normal case, the time-stamp is automatically set to current time of the miner's local system. However, the miner has the ability to alter this value by 900 sec approximately and still manages to convince the other miners the authenticity of the block. This dependency can be exploited in the following scenario. Let's say a seller is trying to sell an option contract and many buyers are interested in entering the contract. The miner can differentiate between various transactions from a list of buyers that arrived after the seller's contract was published by noticing the time-stamp of all the transaction which are greater than that of seller's time. Then, s/he can choose any buyer whose time-stamp is greater than seller's and within the 900 sec period. The miner benefits from the ether that each buyer is offering to the miner to accept his/her transaction.

3) Mishandled Exceptions: In Ethereum, one contract can call another contract using different methods like send( ) instruction or calling a contract's public methods. If any exception or interruption occurs during the execution of these methods, the contract sending the instructions receive an error and the state reverts to its previous stable state but based on how the call was made, the error may or may not reach to the contract, which was supposed to receive that contract. For instance, a buyer of an option contract may call a seller's method to enter the option contract but due to insufficient balance the buyer will not enter the contract. However, if the exception raised due to buyer's insufficient funds did not reach the seller, then s/he may tend to close the contract believing that the buyer has already entered the contract.

4) Re-entrance Vulnerability: It is a well known issue which led to the popular *The DAO* (decentralized autonomous organization) hack [13]. This problem arises when one contract calls another contract and waits the call instruction to finish. For example, first contract wants to pay the second contract some amount of ether and tries to call the second contract to deposit money. Now the malicious owner of the second contract may make use of the intermediate state of first contract and call its own contract several times until the balance gets depleted or the gas used for executing the contract becomes zero.

5) Integer Overflow/ Underflow: Balance in Ethereum are usually represented by an unsigned integer of 256 bits. However, if the balance goes out of this range it will either get an arbitrarily large value or may come down to zero. In either case, the person may suffer huge losses if these vulnerabilities are not handled properly.

For our study, we have developed four versions of European call option contract, each being an advancement over the previous one. The results generated using Oyente [11] for the initial and the final version are depicted in table I.

In the initial version of the contract developed, default values were set for the price at expiration time without actually fetching data from real market. Whereas in the final version, the contract uses *pricegeth* to fetch the feed i.e. the price at the expiration time. In this version, a few bugs in the code were fixed.

Table I
VULNERABILITIES IDENTIFIED USING OYENTE

Vulnerability Type	**Initial Results**	**Final Results**
EVM Code Coverage	45.8%	44.7%
Integer Underflow	True	True
Integer Overflow	True	True
Parity Multisig Bug 2	False	False
Callstack Depth Attack Vulnerability	False	False
Transaction-Ordering Dependence (TOD)	False	True
Time-stamp Dependency	False	False
Re-Entrance Vulnerability	False	False

The final version was executed using the JavaScriptVM environment in Remix IDE. When the contract is executed, some ether are used from the account as gas for contract creation.

Then, on calling one of the functions called buyCallOption, the input for calling the option included the amount of *wei* equal to the call options premium. With the main contract having sufficient funds to pay the buyer of the contract at the expiration. The transaction was completed successfully.

If the function buyCallOption is executed without passing any value, then the transaction fails as the premium has to be paid by the buyer.

If the sender wants to be a writer of the call option, then s/he can call the writeCallOption() function. On calling this functions, with the input as the amount of *wei* equal to the call options default value to be used for settlement at the expiration date, the transaction was successful. If the sender is a writer of an option then the premium will be sent to the sender from the contract.

Results from execution of one of the functions (Write Call Option) is illustrated below in Figure - 1

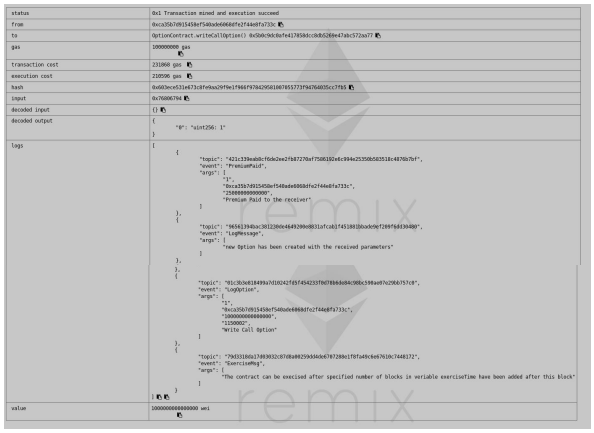


Figure 1. Successful Transaction for Write Call Option

IV. Conclusions

The main objective of this research is to study process of creation of a European Option Contract on a decentralized platform such as Ethereum Blockchain. Two main contribution of this study are (i) Creating a Smart Contract for depicting the functionality of an European Option (ii) and Analyzing the developed contract using a pre-existing tool Oyente.

In our experiments, we tested different possible scenarios which include i) user sending sufficient premium or not for holding/buying a call option. Also, if the contract has sufficient balance to be fixed for settlement ii) user sending enough money to be fixed for settlement of the contract or not being a writer of the call option. These tests were performed by running the contract on Javascript VM environment of the Remix IDE. Further, to explore the potential vulnerabilities in our contract we analyzed it using Oyente and as shown in results, we were able to identify the potential attacks which can happen on our contract. In the case of financial smart contracts, some of the essential conditions required to properly implement the contract on Blockchain are not currently available and hence are depicted as flaws in Oyente. Therefore, we actually need to look into this from a broader perspective and identify that whether the vulnerabilities should actually be avoided or the tool needs to be modified to incorporate specific financial situation, which we leave as future work.

Acknowledgements

The last three authors acknowledge Natural Sciences and Engineering Research Council (NSERC) Canada for partial financial support for this research through Discovery Grants. The first three authors acknowledge the International Graduate Student Scholarship from Faculty of Graduate Studies, University of Manitoba. The third author acknowledges the GETS funding from Faculty of Graduate Studies as well.

References

[1] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. *https://bitcoin.org/bitcoin.pdf*, 2008.

[2] Vivek Vishnumurthy, Sangeeth Chandrakumar, and Emin Gun Sirer. Karma: A secure economic framework for peer-to-peer resource sharing. In *Workshop on Economics of Peer-to-Peer Systems*, volume 35, 2003.

[3] Nick Szabo. Formalizing and securing relationships on public networks. *First Monday*, 2(9), 1997.

[4] Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum Project Yellow Paper*, 151:1–32, 2014.

[5] Karthikeyan Bhargavan, Antoine Delignat-Lavaud, Cédric Fournet, Anitha Gollamudi, Georges Gonthier, Nadim Kobeissi, Natalia Kulatova, Aseem Rastogi, Thomas Sibut-Pinote, Nikhil Swamy, and Santiago Zanella-Bguelin. Formal verification of smart contracts: Short paper. In *Proceedings of the 2016 ACM Workshop on Programming Languages and Analysis for Security*, pages 91–96. ACM, 2016.

[6] Benjamin Egelund-Müller, Martin Elsman, Fritz Henglein, and Omri Ross. Automated execution of financial contracts on blockchains. *Business & Information Systems Engineering*, 59(6):457–467, 2017.

[7] Kevin Delmolino, Mitchell Arnett, Ahmed Kosba, Andrew Miller, and Elaine Shi. Step by step towards creating a safe smart contract: Lessons and insights from a cryptocurrency lab. In *International Conference on Financial Cryptography and Data Security*, pages 79–94. Springer, 2016.

[8] Loi Luu, Jason Teutsch, Raghav Kulkarni, and Prateek Saxena. Demystifying incentives in the consensus computer. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, pages 706–719. ACM, 2015.

[9] Etherdice smart contract. *http://etherdice.io/*, aug 2015. Accessed on April 15, 2018.

[10] J. C. Hull. *Options, Futures and Other Derivatives, 9th edition*. Prentice Hall, 2014.

[11] Loi Luu, Duc-Hiep Chu, Hrishi Olickel, Prateek Saxena, and Aquinas Hobor. Making smart contracts smarter. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS '16, pages 254–269, New York, NY, USA, 2016. ACM.

[12] Nicola Atzei, Massimo Bartoletti, and Tiziana Cimoli. A survey of attacks on ethereum smart contracts (sok). In *International Conference on Principles of Security and Trust*, pages 164–186. Springer, 2017.

[13] The dao attack. *https://www.coindesk.com/understanding-dao-hack-journalists/*, may 2016. Accessed on April 15, 2018.

[14] Schneider J, Blostein A, Lee B, Kent S, Groer I, Beardsley E (2016) Blockchain-putting theory into practice. http://www.finyear.com/attachment/690548/. Online: Accessed; 2018-04-19.

[15] José Parra Moyano and Omri Ross. KYC optimization using distributed ledger technology. *Business & Information Systems Engineering*, 59(6):411–423, 2017.

[16] Patrick Bahr, Jost Berthold, and Martin Elsman. Certified symbolic management of financial multi-party contracts. *ACM SIGPLAN Notices*, 50(9):315–327, 2015.

[17] Iddo Bentov, Alex Mizrahi, and Meni Rosenfeld. Decentralized prediction market without arbiters. In *International Conference on Financial Cryptography and Data Security*, pages 199–217. Springer, 2017.

[18] Meni Rosenfeld. Overview of colored coins. https://bitcoil.co.il/BitcoinX.pdf, 2012.

[19] Oraclize. http://docs.oraclize.it/. Online: Accessed; 2018-04-18.

[20] Buterin Vitalik (2014) SchellingCoin: A Minimal-Trust Universal Data Feed. https://goo.gl/w2aJwu. Online: Accessed; 2018-04-18.

[21] Shayan Eskandari, Jeremy Clark, Vignesh Sundaresan, and Moe Adham. On the feasibility of decentralized derivatives markets. In *International Conference on Financial Cryptography and Data Security*, pages 553–567. Springer, 2017.

[22] Solidity. https://solidity.readthedocs.io/en/v0.4.22/. Online: Accessed; 2018-04-18.

[23] Remix - solidity ide. http://remix.readthedocs.io/en/latest/. Online: Accessed; 2018-04-18.

[24] Pricegeth. https://github.com/VelocityMarket/pricegeth. Online: Accessed; 2018-04-18.

[25] Etherbeat. https://github.com/scorelab/EtherBeat. Online: Accessed; 2018-04-18.

[26] Ethereum foundation. block validation algorithm. *https://github.com/ethereum/wiki/wiki/Block-Protocol-2.0*, feb 2017. Accessed on April 15, 2018.